

This exam consists of 17 questions.

Points are indicated for each question and broken down to each part.

Question 1: Software Engineering (3 points)

List two aspects of how software engineering is different from other engineering disciplines. Illustrate each difference on an example.

Marking:

Lo1, Slide 31

1 mark for listing two aspects, e.g., No fundamental theory, Ease of change, Rapidly evolving technology, Negligible manufacturing cost, No borders

1 mark each for each illustrated reason (0 marks if software engineering is illustrated as a whole, but not the reason)

Question 2: Simple Programs vs. Software Products (6 points)

List and briefly explain three aspects from the lecture how simple *programs* are different from *software products*. For each aspect explain if and how SCRUM reflects the difference.

Marking:

Lo1, Slide 31 and Lo8, Slide 42-52

1 mark x 3 per difference with explanation (examples here just list properties), e.g., Typically small, Author is sole user, Single author, No proper user interface, No proper documentation, Ad hoc development vs. Large, Large number of users, Team of developers, Well-designed interface, Well documented, Systematic development

1 mark x 3 for each how it is addressed, e.g., no proper user interface – customer onsite to check prototype, team of developers – daily stand up in scrum, spring planning and coordination roles

Question 3: Rational Unified Process vs. Waterfall Model (6 points)

Pick three aspects of the Rational Unified Process (RUP), contrast them with the Waterfall model, and explain the intended benefit of each aspect of the RUP.

Marking:

1 mark x 3 for mentioning and matching to corresponding aspect of waterfall model (only 0.5 if aspect is not matched to Waterfall model), examples

- Several iterations of workflows to improve work elements
- workflows executed in each phase for later corrections
- documentation based heavily on models and use cases to avoid ambiguity

1 mark for reasonable benefit

must come in triplets; (RUP, waterfall, benefit), not enough to list one benefit or general ones that do not match the contrasted elements

Question 4: Software Engineering Mistakes (6 points)

List 3 classic software engineering mistakes from the lecture and explain for each how the Rational Unified Process addresses it.

Marking:

1 mark x 3 for each mistake:

- scheduling: short phases with identifying highest risks
- planning issues: see above
- gold plating requirements: many iterations and evaluation, force progress to next issue
- silver bullet: evaluation in every iteration

LO2, Slide 44-68

1 mark x 3 for the mitigation of each mistake. Mitigation has to be specific to each point (this is not a recall, but a transfer task)

Question 5: Stakeholders (4 points)

Name four stakeholders (at least two external stakeholders) for a software for a car repair shop and provide for each a relevant concern for the software.

Marking:

1 mark for every stakeholder and relevant concern

if no concerns are given, at most one mark in total

- Developers need specification and have to develop software
- waiters need to know who ordered what
- chefs want to know specific choices of variation of food
- customers want information about food
- banks want to ensure safe payments

Question 6: Requirements classification (4 points)

For each of the following requirements, mark if they are functional (F) or non-functional (N):

- (a) N The system should support concurrent access by at least 1000 users.
- (b) F The system should process online payments.
- (c) N The system should encrypt sensitive customer data, such as credit card information.
- (d) N The system's database should be backed up every night at 2:00 AM.

Question 7: User stories (4 points)

Write two user stories as requirements for a car repair shop system.

Marking:

2 marks for each, where -1 mark for missing any of the three elements: role, capability, value

Example: "As a student I want to be able to compare visa types so that I can make the correct appointment for the required visa."

0 if it is not a user story

Question 8: Use Case Diagram vs. Class Diagram (6 points)

List two commonalities and two differences between Use Case Diagrams and Class Diagrams. Give an example where one diagram type would be used over the other.

Marking:

1 mark for each commonality, e.g., both have inheritance, both have associations

1 mark for each difference, e.g., system boundaries explicit in UC diagram, but not in CD, messages actors are in UC but not CD

2 marks for the example: 1 mark for relevance, 1 mark for correctness of choice of diagram

Question 9: Incompleteness (4 points)

Define incompleteness in requirements engineering. Give an example requirement where incompleteness cannot be avoided and explain why.

Marking:

Lo3, Slide 13, elicitation problems Slide 39

1 mark what it means "common that some requirements are missing, and so the system eventually lacks some important functionality", difficulties of elicitation

Lo4, Slide 17-18

1 mark for the example, 2 marks for explanation

Question 10: Modeling Language Elements (6 points)

List and explain (max. one sentence) the three elements of a modeling language definition. Illustrate them on feature models.

Marking:

1 mark per element and 1 mark per explanation

- syntax
- semantics
- semantic mapping

only 1 mark in total if illustration on feature models

Question 11: Implementation guidelines (6 points)

List three guidelines to follow during the implementation of a system and explain for each how it supports the development process.

Marking:

Lo6, Slide 48

1 mark for each guideline, e.g., using version control, commenting code, following coding styles, writing comments, bug tracking system use

1 mark for explanation of how each helps (must be specific to guideline)

Question 12: Testing techniques (4 points)

Describe one scenario where black-box testing and one scenario where white-box testing should be applied. Justify each answer.

Marking:

1 mark per scenario

1 mark per justification

Question 13: Combinatorial test design (8 points)

Use the In-Parameter-Order algorithm to complete the test plan below for pairwise coverage ($t = 2$). The parameters are $A = \{a_1, a_2\}$, $B = \{b_1, b_2\}$, and $C = \{c_1, c_2, c_3\}$.

a_1	b_1
a_1	b_2
a_2	b_1
a_2	b_2

Marking:

- 1 mark for each element added to existing rows
- 2 marks each for rows (ranges from 0-2, since it is a greedy algorithm)
- 1 for each missing pair (pair of values that is not covered)
- 0 if the algorithm was not followed, e.g., just listing tuples

Question 14: Metamorphic testing (4 points)

Explain how an oracle can be (partially) constructed using metamorphic testing on the example of an exam grading system. Clearly state the metamorphic relation you assume to hold.

Marking:

- 2 marks the oracle, e.g., more answers lead to more points (or correct vs incorrect ones)
- 2 marks for how to use it to compare two test outputs, e.g., feed one exam to the grading system, feed the same exam with some answers deleted to the same system, compare the scores to make sure the first is greater or equal to the second

Question 15: White-box Testing

- (a) (6 points) Draw the control flow graph for the given code snippet (the lines are spaced to draw the graph next to the code). Labeling nodes with the line number is sufficient.

```
1 public static boolean check(int x) {  
2     if (x <= 3) {  
3         return false;  
4     }  
5     boolean y = true;  
6     for (int i = 0; i < x && y; i++) {  
7         if (x % i == 0) {  
8             y = false;  
9         }  
10    }  
11    return y;  
12 }
```

Marking:

Log, Slide 11-18

- 1 for each wrong edge and each missing node (down to 0/6)

- (b) (2 points) Provide a minimal set of test cases for 100% statement coverage.

Marking:

Example: (x=1); (x=4); (-1 if not minimal)

We need at least two because one needs to use the return in line 3 and one in line 11.

- (c) (3 points) Provide a minimal set of test cases for 100% branch coverage.

Marking:

Example: (x=1); (x=9); 1.5 marks each (-1 if not minimal)

- (d) (2 points) Explain the relation between branch coverage and MC/DC coverage.

Marking:

MC/DC = Modified condition/decision coverage subsumes branch coverage (=decision coverage)

(e) (2 points) Compute the cyclomatic complexity of the program.

Marking:

based on control flow graph: $\#Edges - \#Nodes + 2 \times \#Components$

-1 if formula incorrect

-1 if computation incorrect

(computation must be based on control flow graph provided in (a))

Question 16: Extreme Programming (4 points)

Name and briefly describe two practices of Extreme Programming.

Marking:

Lo8, Slide 19-40

1 mark x2 per practice named: Incremental planning, Small releases, Simple design, Test first, Refactoring, Pair programming, Continuous integration, On-site customer

1 mark x2 per explanation

Question 17: Code Quality (4 points)

Define the Law of Demeter. Illustrate a violation and state why violations may be a problematic.

Marking:

Lo9, Slide 26

1 mark for definition "Only talk to your direct friends"

2 marks for illustration

1 mark for explanation